

```
test_app.sub_app - ERROR - Debug message from sub_app
test_app - INFO - After calling the sub_app_function
% cat file.log
2021-11-28 23:26:04,332 - test_app - WARNING - This is a
warning from main module
2021-11-28 23:26:04,332 - test_app - ERROR - This is an error
from main module
2021-11-28 23:26:04,332 - test_app.sub_app - ERROR - Debug
message from sub_app
```

Logging from multiple threads

Logging from multiple threads is as simple as the normal one. To differentiate the logs generated from each thread, `threadName` can be used. An example of logging from the main thread and another thread is shown below.

```
import logging
import threading
import time

def worker(arg):
    while not arg['stop']:
        logging.debug('Hi from thread 2')
        time.sleep(0.5)

def main():
    logging.basicConfig(level=logging.DEBUG,
        format='%(asctime)s - %(name)s - %(levelname)s -
        %(threadName)s - %(message)s')
    info = {'stop': False}
    thread = threading.Thread(target=worker, args=(info,))
    thread.start()
    while True:
        try:
            logging.debug('Hello from main thread')
            time.sleep(1.0)
        except KeyboardInterrupt:
            info['stop'] = True
            break
    thread.join()
if __name__ == '__main__':
    main()
```

The output is:

```
% python threadlogger.py
2021-11-28 22:38:54,207 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:54,207 - root - DEBUG - MainThread - Hello
from main thread
2021-11-28 22:38:54,712 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:55,212 - root - DEBUG - MainThread - Hello
from main thread
```

```
2021-11-28 22:38:55,214 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:55,716 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:56,217 - root - DEBUG - MainThread - Hello
from main thread
2021-11-28 22:38:56,220 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:56,725 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:57,221 - root - DEBUG - MainThread - Hello
from main thread
2021-11-28 22:38:57,229 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:57,734 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:58,223 - root - DEBUG - MainThread - Hello
from main thread
2021-11-28 22:38:58,236 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:58,738 - root - DEBUG - Thread-1 - Hi from
thread 2
2021-11-28 22:38:59,226 - root - DEBUG - MainThread - Hello
from main thread
2021-11-28 22:38:59,239 - root - DEBUG - Thread-1 - Hi from
thread
```

Context logging and filter

- **LoggerAdapter:** Sometimes the logging output is needed to contain contextual information in addition to the parameters passed to the logging call. For example, in a networked application, it may be desirable to log client-specific information in the log (e.g., remote client's user name, or IP address). An easy way in which you can pass contextual information to the output along with the logging event information is to use the *LoggerAdapter* class. In the example that follows we are adding the *host_ip* to all the log events.
- **Filter:** A filter can be used for both the logger and handler levels, as it can stop certain information from being passed. It will also inject additional information into the *LogRecord*, which will be logged. In our example below, we are using filters to allow only certain levels to be logged. Though the log level is set to 'debug', we are only allowing 'info' and 'error' to be logged.



Note: These are fictional examples, as use cases in real life vary.

Continued to page no...66